SYS3 PRACTICALS

BRIEFING LECTURE

SYS3 Practicals

Topic Focus

- Instruction sequencing and pipelining
- Using Pipeline Emulator C++ library

- So do I need to know C++ ?
  - **no**, only a very minimal amount of C++ needed

SYS3 Practicals

Pipelines

Pipelines represent stages of instruction execution.

Example:

ADD R1,R2,R3 (R3 ← R1+R2)

[--**IF**--] [--**ID**--] [**RR..R1**] [**RR..R2**] [--**IA**--] [**WB..R3**]

SYS3 Practicals

Example:

ADD R1,R2,R3 (R3 ← R1+R2)

[--**IF**--] [--**ID**--] [**RR..R1**] [**RR..R2**] [--**IA**--] [**WB..R3**]

SYS3 Practicals

Example:

ADD R1,R2,R3 (R3 ← R1+R2)

[--**IF**--] [--**ID**--] [**RR..R1**] [**RR..R2**] [--**IA**--] [**WB..R3**]

Instruction Fetch

SYS3 Practicals

Example:

ADD R1,R2,R3 (R3 ← R1+R2)

[--**IF**--] [--**ID**--] [**RR..R1**] [**RR..R2**] [--**IA**--] [**WB..R3**]

Instruction Decode

Instruction Fetch

SYS3 Practicals

Example:

ADD R1,R2,R3 (R3 ← R1+R2)

[--**IF**--] [--**ID**--] [**RR..R1**] [**RR..R2**] [--**IA**--] [**WB..R3**]

Register Read x 2

Instruction Decode

Instruction Fetch

SYS3 Practicals

Example:

ADD R1,R2,R3 (R3 ← R1+R2)

[--**IF**--] [--**ID**--] [**RR..R1**] [**RR..R2**] [--**IA**--] [**WB..R3**]

Integer ALU cycle

Register Read x 2

Instruction Decode

Instruction Fetch

SYS3 Practicals

Example:

ADD R1,R2,R3 (R3 ← R1+R2)

[--**IF**--] [--**ID**--] [**RR..R1**] [**RR..R2**] [--**IA**--] [**WB..R3**]

Register Write

Integer ALU cycle

Register Read x 2

Instruction Decode

Instruction Fetch

SYS3 Practicals

Example:

ADD R1,R2,R3 (R3 ← R1+R2)

[--**IF**--] [--**ID**--] [**RR..R1**] [**RR..R2**] [--**IA**--] [**WB..R3**]

6 clock cycles

CPI = 6.0
IPC = 1/6.0 = 0.167

SYS3 Practicals

Example (sequential execution):

ADD R1,R2,R3 (R3 ← R1+R2)

ADD R3,R4,R5 (R5 ← R3+R4)

[--IF--] [--ID--] [RR..R1] [RR..R2] [--IA--] [WB..R3] [--IF--] [--ID--] [RR..R3] [RR..R4] [--IA--] [WB..R5]

12 clock cycles, 2 instructions

CPI = 12/2 = 6.0
IPC = 1/6.0 = 0.167

SYS3 Practicals

Example (pipelined execution):

ADD R1,R2,R3 (R3 ← R1+R2)

ADD R3,R4,R5 (R5 ← R3+R4)

```
[----IF----] [----ID----] [RR..R1] [RR..R2] [---IA----] [WB..R3]
```

```
[----IF----] [---ID----] [RR..R3] [RR..R4] [----IA---] [WB..R5]
```

7 clock cycles, 2 instructions

CPI = 7/2 = 3.5 → **Speedup** : 6.0/3.5 =1.71

SYS3 Practicals

Example (Hazard):

```
[----IF----] [----ID----] [RR..R1] [RR..R2] [---IA----] [WB..R3]
             [----IF----] [---ID----] [RR..R3] [RR..R4] [----IA---] [WB..R5]
```

RAW (Read After Write) Hazard

– R3 is read before its previous value is written

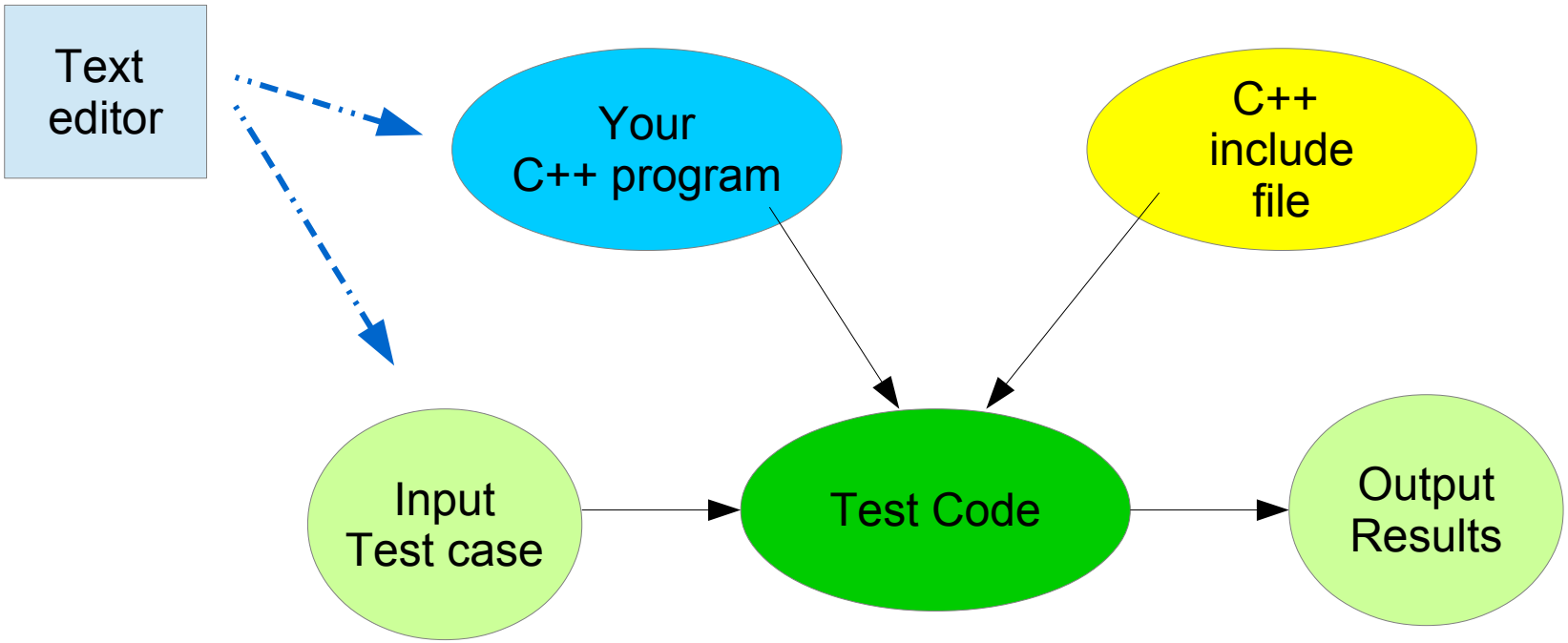
– This is an error unless it is fixed.

– It happens because we have pipelined the stages

Tools and activities

SYS3 Practicals

- What do we do in the practicals?



SYS3 Practicals

- Tool output

Your test program machine code

Pipelined code

```
---- ASM CODE LISTING ----------
0: LDR - R1 - R5 -
1: LDR - R5 - R9 -
2: LDI - R9 - 992 -
3: LDI - R2 - 221 -
4: IADD - R2 - R6 - R5

---- PIPELINE DIAGRAM----
000:     LDR.     R1.     R5.          >> [I=000]  [--IF--][--ID--][RR..R1][--OF--][WB..R5]
001:     LDR.     R5.     R9.          >> [I=001]  [   --  ][--IF--][--ID--][RR..R5][--OF--][WB..R9]
002:     LDI.      R9.    992.         >> [I=002]  [  --  ][  --  ][--IF--][--ID--][--OF--][WB..R9]
003:     LDI.      R2.    221.         >> [I=003]  [  --  ][  --  ][  --  ][--IF--][--ID--][--OF--][WB..R2]
004:    IADD.     R2.     R6.    R5  >> [I=004]   [  --  ][  --  ][  --  ][  --  ][--IF--][--ID--][RR..R2][RR..R6][--
IA--][WB..R5]
```

SYS3 Practicals

- Tool output

```
---- PIPELINE DIAGRAM----
000:     LDR.      R1.      R5.          >> [I=000]  [--IF--][--ID--][RR..R1][--OF--][WB..R5]
001:     LDR.      R5.      R9.          >> [I=001]  [  --  ][--IF--][--ID--][RR..R5][--OF--][WB..R9]
002:     LDI.      R9.     992.          >> [I=002]  [  --  ][  --  ][--IF--][--ID--][--OF--][WB..R9]
003:     LDI.      R2.     221.          >> [I=003]  [  --  ][  --  ][  --  ][--IF--][--ID--][--OF--][WB..R2]
004:    IADD.      R2.      R6.      R5  >> [I=004]  [  --  ][  --  ][  --  ][  --  ][--IF--][--ID--][RR..R2][RR..R6][--IA--][WB..R5]

                                                     [C-0000][C-0001][C-0002][C-0003][C-0004][C-0005][C-0006][C-0007][C-0008][C-0009]
```

On the command line, the tabular output is properly aligned as above

SYS3 Practicals

- Tool output

```
-- [ STATS ] ----------------------------------------

 Instruction Count    5
 Sequential Cycles   24
 Pipelined  Cycles    9
 Speedup          2.67
 Sequential CPI       4.80
 Pipelined  CPI       1.80

-- [ MEMORY HAZARDS (Unified Cache) ] ----------------

 Memory Hazard(s) at Instruction 3 Cycle 3 [--IF--]
 Memory Hazard(s) at Instruction 4 Cycle 4 [--IF--]

-- [ REGISTER HAZARDS ] ----------------

 RAW Hazard ? --  RR.2(i4,c6)) WB.2,(i3,c6)
 WAW Hazard ? --  RR.9(i2,c5)) WB.9,(i1,c5)

-- [ STRUCTURAL HAZARDS ] ----------------

 WRITE PORTS Exceeded at c05 (2 used vs 1 available)
```

Statistics & performance data

Pipelined hazards detected

The tool will detect most of the hazards we are interested in and give some details about where they are located.

SYS3 Practicals

- Tool output

```
---- PIPELINE DIAGRAM----
000:     LDR.      R1.      R5.          >> [I=000]  [--IF--][--ID--][RR..R1][--OF--][WB..R5]
001:     LDR.      R5.      R9.          >> [I=001]  [  --  ][--IF--][--ID--][RR..R5][--OF--][WB..R9]
002:     LDI.      R9.     992.          >> [I=002]  [  --  ][  --  ][--IF--][--ID--][--OF--][WB..R9]
003:     LDI.      R2.     221.          >> [I=003]  [  --  ][  --  ][  --  ][--IF--][--ID--][--OF--][WB..R2]
004:    IADD.      R2.      R6.      R5  >> [I=004]  [  --  ][  --  ][  --  ][  --  ][--IF--][--ID--][RR..R2][RR..R6][--IA--][WB..R5]

                                                     [C-0000][C-0001][C-0002][C-0003][C-0004][C-0005][C-0006][C-0007][C-0008][C-0009]
```

RAW Hazard ? -- RR.2(i4,c6) WB.2,(i3,c6)

The PipelineClass C++ Class

SYS3 Practicals

PipelineClass

A C++ class is somewhat simplistically similar to a struct in C.

- Variables can exist within the class
- Functions can also exist

- Classes exists as instances, e.g. Pipe could be an instance of PipelineClass.

SYS3 Practicals

```
PipelineClass  Pipe;    // declare a class inst.

Pipe.IssueWidth = 3;  // modify a variable

Pipe.SomeFunc(...);   // call a class function
```

This is just about all you need to know about C++ for the practicals.

SYS3 Practicals

PipelineClass – includes …

- Variables relating to CPU configuration
- Functions to load assembler files
- Functions to generate pipeline diagrams
- Functions to detect the state of a pipeline stage

You need to read the documentation !

```
int main(int argc, char * argv[])
{

    int c=0;

    Pipe.IssueWidth=4;

    Pipe.StartupMessage();

    Pipe.ReadAssemblerCode(argv[1]);
    Pipe.DumpCodeList();

    Pipe.InitialSchedule();

    Pipe.DumpPipeline();
    Pipe.PipelineTest();

    printf("\n---- Rescheduling Code ------\n");

    c = FixMemoryHazards();

    printf("-Detected and fixed %d Memory Hazards\n",c);

    Pipe.DumpPipeline();
    Pipe.PipelineTest();

}
```

Configuring CPU parameters

```
int main(int argc, char * argv[])
{

    int c=0;

    Pipe.IssueWidth=4;

    Pipe.StartupMessage();

    Pipe.ReadAssemblerCode(argv[1]);
    Pipe.DumpCodeList();

    Pipe.InitialSchedule();

    Pipe.DumpPipeline();
    Pipe.PipelineTest();

    printf("\n---- Rescheduling Code ------\n");

    c = FixMemoryHazards();

    printf("-Detected and fixed %d Memory Hazards\n",c);

    Pipe.DumpPipeline();
    Pipe.PipelineTest();

}
```

```
int main(int argc, char * argv[])
{

    int c=0;

    Pipe.IssueWidth=4;

    Pipe.StartupMessage();

    Pipe.ReadAssemblerCode(argv[1]);
    Pipe.DumpCodeList();

    Pipe.InitialSchedule();

    Pipe.DumpPipeline();
    Pipe.PipelineTest();

    printf("\n---- Rescheduling Code ------\n");

    c = FixMemoryHazards();

    printf("-Detected and fixed %d Memory Hazards\n",c);

    Pipe.DumpPipeline();
    Pipe.PipelineTest();

}
```

Generate pipeline diagram
And test for hazards

```
int main(int argc, char * argv[])
{

    int c=0;

    Pipe.IssueWidth=4;

    Pipe.StartupMessage();

    Pipe.ReadAssemblerCode(argv[1]);
    Pipe.DumpCodeList();

    Pipe.InitialSchedule();

    Pipe.DumpPipeline();
    Pipe.PipelineTest();

    printf("\n---- Rescheduling Code ------\n");

    c = FixMemoryHazards();

    printf("-Detected and fixed %d Memory Hazards\n",c);

    Pipe.DumpPipeline();
    Pipe.PipelineTest();

}
```

Custom code (you write this)

```
int main(int argc, char * argv[])
{

    int c=0;

    Pipe.IssueWidth=4;

    Pipe.StartupMessage();

    Pipe.ReadAssemblerCode(argv[1]);
    Pipe.DumpCodeList();

    Pipe.InitialSchedule();

    Pipe.DumpPipeline();
    Pipe.PipelineTest();

    printf("\n---- Rescheduling Code ------\n");

    c = FixMemoryHazards();

    printf("-Detected and fixed %d Memory Hazards\n",c);

    Pipe.DumpPipeline();
    Pipe.PipelineTest();

}
```

Show new pipeline state
And test for hazards again

SYS3 Practicals

Next Steps:

- **First lab**
  - familiarising with the tool
  - Understand a simple C/C++ test program
- **Subsequent labs**
  - Understanding specific hazards
  - writing algorithms to find and fix hazards

SYS3 Practicals

Next Steps:

- **First lab**
  - familiarising with the tool
  - Understand a simple C/C++ test program
- **Subsequent labs**
  - Understanding specific hazards
  - writing algorithms to find and fix hazards